



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Lab Report On Basic File and Directory Operations in Linux

Course: Operating System and System Programming Lab

Course Code: CSE 3102

Submission Date: 28/05/2025

Submitted by	Submitted to
Name: Adnan Zaman Niloy Student ID: 210142 Year: 3rd Semester: 1 st Session: 2021-22	Monishanker Halder, Assistant Professor, Department of Computer Science and Engineering, Jashore University of Science and Technology

Experiment Name:

Basic File and Directory Operations in Linux

Introduction:

Linux is a widely used open-source operating system that supports command-line interface (CLI) operations. Understanding basic file and directory management commands is essential for efficiently working with Linux systems. In this experiment, we practiced fundamental operations such as creating, moving, copying, and deleting files and directories using terminal commands.

Methodology

The following Linux terminal commands were used to perform basic file and directory operations. Each command was applied in a sequence to create, navigate, and manage files and folders effectively within a project structure. Below are the details:

The following **Linux commands** were used in the terminal to perform basic file and directory operations as part of organizing a project folder named **OSlab**.

1. **pwd**

This command helps to check the current location in the file system, which is useful when navigating through multiple directories.

It displays the **full path** of the current working directory.

Example: pwd

Output: /home/user/OSlab

2. **mkdir <directory_name>**

This is used to create a new folder to keep project files organized and separate from other data.

It creates a **new directory** with the name provided.

Example: mkdir OSlab

This creates a folder named **OSlab**.

3. **cd <directory_name>**

After creating a folder, this command is used to go inside it so you can start working there.

It changes the **current directory** to the specified one.

Example: cd OSlab

Now you're working inside the **OSlab** folder.

4. **touch <filename>**

This is used to create a new empty file that can later be filled with content such as text or code.

It creates a **new file** or updates the timestamp of an existing one.

Example: touch Studentinformation.text

This creates a file named **Studentinformation.text** inside the **OSlab** folder.

5. **ls**

Before or after creating files and folders, this command is used to check what items exist in the current directory.

It lists all **files and subdirectories** in the current folder.

Example: ls

This will show files like **Studentinformation.text**.

6. **rm <filename>**

When a file is no longer needed, this command deletes it permanently from the system.

It **removes** the specified file.

Example: rm temp.txt

This deletes the file named **temp.txt**.

7. **cat <filename>**

This command is used to quickly read the content of a file directly in the terminal.

It displays the **contents** of a text file.

Example: cat Studentinformation.text

It shows the contents of the file **Studentinformation.text**.

8. **mv <source> <destination>**

This command is used either to **rename** a file or to **move** it to another location.

It moves or renames files and directories.

Example: mv Studentinformation.text Info.text

This renames the file to **Info.text**.

9. **cp <source> <destination>**

To keep a backup or make a duplicate, this command is used to copy a file.

It copies the file from one location to another.

Example: cp Studentinformation.text OSlab1.text

This makes a copy named **OSlab1.text** in the same directory.

10. **cp -r <source> <destination>**

This command is useful when an **entire folder**, including its files and subfolders, needs to be copied.

It **recursively copies directories** and their contents.

Example: cp -r OSlab OSlab_Backup

This makes a full copy of the **OSlab** folder and names it **OSlab_Backup**.

11. **cd ~**

When finished working, this command quickly takes you back to your **home directory**.

It navigates to the user's home folder.

Example: `cd ~`

You return to **/home/user**.

12. `cd ..`

If you want to go one level up to the **parent directory**, this command is used.

It moves you up one directory in the structure.

Example: `cd ..`

If you're in **/home/user/OSlab**, it takes you to **/home/user**.

Practical Example:

First, I created a folder named **OSlab** using `mkdir OSlab`.

Then, I went into the folder with `cd OSlab`.

I created a file called **Studentinformation.text** using `touch Studentinformation.text`.

To make a copy of that file, I used `cp Studentinformation.text OSlab1.text`.

Now both **Studentinformation.text** and **OSlab1.text** are present in the **OSlab** folder.

Result & Discussion:

All the commands executed successfully. We observed the following:

- Directory creation and navigation allowed structured file management.
- File creation and manipulation using `touch`, `cat`, `cp`, and `rm` were effective.
- The `mv` command was used to rename a directory (e.g., `OSlab` to `OSLab`).
- Copying directories with `cp -r` maintained the folder structure.
- Mistyped commands like `tuch` and `cd~` returned errors, highlighting the importance of correct syntax.

These results confirm the effectiveness of Linux commands in managing the filesystem.

Conclusion:

This experiment helped in understanding and practically applying basic Linux commands for file and directory management. Mastery of these commands is essential for any Linux user, especially system administrators and developers. Regular practice can help avoid common mistakes and improve efficiency in navigating and managing Linux systems.